

课程学习小组讨论记录

【讨论主题】：AI 对软件开发的影响及工具分享

【课程名称】：软件项目开发与实践

【讨论日期】：2026 年 6 月

【讨论时长】：2 小时

【主持人】：谢宇鹏

【参与人员】：谢宇鹏（学号：5120243390）

第 1 部分 讨论背景与目标

【讨论背景】

最近两年，AI 编程工具发展得特别快。最早的时候，AI 只能帮你补全几个单词、几行代码，就像手机打字时的联想功能一样。但到了 2024-2025 年，Cursor、Claude Code、Trae 这些新工具出来了，AI 的能力有了质的飞跃——它不再只是帮你补代码了，而是能理解你想做什么，帮你规划怎么做、帮你写出完整的功能模块、帮你找 Bug 修 Bug。

我们这门课用的是 B 模式（AI 智能协作开发），就是说游戏项目要在 AI 的辅助下来完成。这不只是「让 AI 帮我写代码」那么简单——我们需要理解 AI 擅长什么、不擅长什么，学会怎么跟 AI「合作」，让它成为我们的好帮手而不是替代品。

我们做的「星露谷战斗模块」项目，从需求讨论到代码实现，全程都有 AI 参与。这次讨论就是来总结：AI 到底怎么影响了软件开发？我们用了哪些 AI 工具？怎么跟 AI 配合才能做出好东西？

【讨论目标】

1. 分析 AI 对软件开发各个阶段（想需求、做设计、写代码、找 Bug）带来了什么变化
2. 对比我们用过的几款 AI 工具，说说各自的好坏
3. 总结跟 AI 合作时的有效方法和技巧
4. 讨论「怎么给 AI 下指令」这件事为什么重要
5. 聊聊 AI 时代程序员的角色会怎么变

第 2 部分 主要讨论内容

AI 怎么影响了软件开发的各个环节

想需求的阶段

- **好处：**你只需要跟 AI 说「我想做一个类似星露谷的打怪小游戏」，它就能帮你展开成一份结构化的需求清单——要有哪些功能、什么场景、哪些角色，帮你从一团模糊的想法变成清晰的列表
- **好处：**AI 能帮你生成需求文档的框架，用专业的格式写出来，省去很多排版的功夫
- **要注意：**AI 对「好不好玩」这种感性的东西理解不太到位，它更偏技术视角，所以游戏性方面还得我们自己判断
- **要注意：**AI 列出来的需求可能太多太杂，得我们自己根据时间和能力来裁剪，不能什么都想要
- **我们怎么做的：**先跟 AI 讨论游戏定位（「做星露谷的战斗模块」），再让 AI 细化功能点（最后列出了 14 个功能需求），然后我们自己调整优先级和细节

做设计的阶段

- **好处：**告诉 AI 你的技术栈和偏好，它能快速帮你画出模块怎么划分、类怎么设计、数据怎么流动
- **好处：**你可以让 AI 比较好几种不同的设计方案，帮你分析每种优缺点
- **要注意：**AI 有时候会「过度设计」——明明简单的东西它给你搞得很复杂，引入不必要的结构
- **要注意：**AI 可能不了解你的具体限制（比如 Pygame 有什么坑、课程给的时间有多少），设计出来的东西可能不接地气
- **我们怎么做的：**告诉 AI 我们要用面向对象、分层架构，让它画出类结构图，然后我们自己审查，把不必要的删掉

写代码的阶段

- **好处：**AI 能快速生成那些模式化的代码（比如精灵类、状态机、碰撞检测的框架），省掉大量重复劳动
- **好处：**AI 对 Pygame 这些库的 API 很熟，调用方式基本不会错

- **好处：**遇到 Bug 的时候，把报错信息发给 AI，它能很快帮你定位问题、给出修复方案
- **要注意：**AI 写的代码可能有逻辑错误，比如碰撞检测的边界情况没处理好
- **要注意：**AI 的代码「能跑但不一定好」——有时候写得很臃肿，需要我们自己重构优化
- **我们怎么做的：**一个模块一个模块地来——先做地图，跑通了再做玩家，再做怪物，再做武器。每做完一个模块就立刻运行测试，发现问题马上反馈给 AI 修

找 Bug 的阶段

- **好处：**AI 能根据功能需求帮你列出测试清单，不用自己一个一个想
- **好处：**把错误堆栈信息发给 AI，它能快速指出哪里出了问题，给出修复建议
- **要注意：**AI 修了一个 Bug 可能又引入了新的 Bug，修完之后得全面测一遍
- **我们怎么做的：**把运行时报错的完整信息复制给 AI，AI 给出修复方案，我们先理解再应用，不是盲目照搬

我们用过的 AI 工具对比

工具	类型	我们怎么用的	好在哪	不好在哪
Claude Code	命令行工具	讨论需求、设计架构、写代码、写文档	理解长文本能力强，适合深入讨论；生成代码质量高；能直接帮你改文件	需要付费
GitHub Copilot	编程助手插件	日常写代码时自动补全	跟编辑器集成流畅，对各种库都很熟，补全速度快	只会补全，不会对各种库都很熟，主动帮你规划
Cursor	智能编辑器	全流程开发、直接编辑文件	体验流畅，能直接创建和修改多个文件，智能程度高	有一定学习成本
Trae	国产智能编辑器	备选方案	免费使用，中文支持好，功能还在不断完善	免费版有次数限制

工具	类型	我们怎么用的	好在哪	不好在哪
ChatGPT	网页对话	技术调研、解释原理	灵活，知识面广，什么都能聊	不实时，代码上下文有限

我们的心得：

- 深入讨论和设计：用 Claude Code 最合适，它能理解很长的上下文，跟你来回讨论
- 日常写代码：GitHub Copilot 或 Cursor，写代码时自动补全，效率很高
- 从头到尾做一个项目：Cursor 或 Claude Code 的 Agent 模式，能直接帮你操作文件
- 预算有限：Trae（免费）+ ChatGPT（免费版）也能用

跟 AI 合作的方法和技巧

想需求的时候怎么跟 AI 聊

1. 从模糊到具体：先说个大概（「做个类似星露谷的打怪游戏」），再一步步细化（「要有哪几种武器？几种怪物？几张地图？」）
2. 让 AI 帮你展开：跟 AI 说「帮我列一个完整的功能清单」，它会帮你生成结构化的列表
3. 自己裁剪：AI 列出来的东西可能太多，根据你的实际能力和时间来决定先做什么后做什么
4. 记下来：把讨论结果保存成文档，后面写代码的时候可以参考

做设计的时候怎么跟 AI 聊

1. 说清楚限制：告诉 AI 你用什么技术、有什么要求、不能用什么（「用 Python + Pygame，面向对象，多文件结构，不用 TMX 地图格式」）
2. 让 AI 解释：不要只让 AI 给代码，让它解释为什么这么设计，你才能学到东西
3. 让 AI 比较：让 AI 列出几种方案，对比各自的优缺点，你再做决定
4. 让 AI 出文档：让 AI 帮你生成设计文档的框架，你再补充技术细节

写代码的时候怎么跟 AI 聊

1. **一个一个来**：不要让 AI 一次性把整个项目写出来，而是先做地图，再做玩家，再做怪物，一个模块一个模块地来
2. **告诉 AI 上下文**：写新模块的时候，告诉 AI 已有的类和接口长什么样，确保新代码能跟旧代码对接
3. **写完就测**：每做完一个模块立刻跑一遍，有问题马上反馈给 AI 修
4. **保存版本**：每次 AI 生成的代码确认没问题后立刻提交版本记录，方便以后回退

找 Bug 的时候怎么跟 AI 聊

1. **给完整信息**：把完整的报错信息复制给 AI，不要只说「报错了」
2. **说清楚期望**：告诉 AI「我期望它怎么做，但实际上它怎么了」，AI 才能准确判断问题
3. **先理解再改**：AI 给出修复方案后，先搞懂为什么这么改，再应用，不要盲目复制粘贴
4. **修完要验证**：修了一个 Bug 后，检查其他功能有没有被影响

怎么给 AI 下指令（提示词技巧）

在 B 模式开发中，你怎么跟 AI 说话，直接决定了 AI 输出的质量。

好的指令长什么样

1. **目标明确**：「请用 Python 实现一个怪物自动绕开障碍物追玩家的寻路功能」比「帮我写个追人功能」好得多
2. **有约束条件**：「在 20×11 的格子地图上，用两个格子之间的距离来估算远近」
3. **提供背景**：「我正在做 Pygame 游戏，已经有一个 Entity 基类，里面有移动和碰撞检测方法」
4. **要求格式**：「请返回完整的 Python 代码，加上中文注释」
5. **迭代改进**：「这段代码能跑了，但太慢了，请优化，减少每帧的计算量」

常见的错误指令

1. **太模糊**：「帮我做个游戏」——AI 不知道做什么类型、什么规模
2. **没约束**：「写个 AI」——没有指定用什么语言、什么框架、做什么功能
3. **一次太多**：「帮我做整个项目」——应该拆成一个个小任务
4. **不给反馈**：AI 第一次输出不满意时，应该说哪里不满意，而不是换个方式重新问

AI 时代程序员的角色变化

从「写代码」到「看代码」

以前程序员的核心工作是自己一行一行写代码。现在 AI 几秒钟就能生成你几个小时才能写完的代码量，所以写代码本身不再是核心竞争力了。程序员的角色更像是「审查员」——判断 AI 写的东西好不好、有没有问题、符不符合需求。

从「做功能」到「做设计」

AI 很擅长做具体的事情（比如「实现一个碰撞检测」），但不太擅长想整体方案（比如「怎么划分模块才能让系统好扩展」）。所以架构设计能力反而变得更重要了——这是 AI 替代不了的。

从「会语法」到「懂原理」

AI 能写出语法正确的代码，但程序员需要理解底层原理（比如为什么要先处理水平碰撞再处理垂直碰撞），才能判断 AI 写的东西合不合理。不懂原理的话，AI 写了个有隐蔽 Bug 的代码你也看不出来。

给 AI 下指令成了新技能

能准确描述需求、把大任务拆成小任务、引导 AI 输出高质量的结果，成了 AI 时代程序员的核心技能之一。这其实跟传统的需求分析能力一脉相承，但要求更高——你不仅要懂业务，还要懂 AI 擅长什么、不擅长什么。

第 3 部分 总结与反思

讨论总结

通过这次深入讨论，我们得出以下核心结论：

1. **AI 是工具不是替代**：AI 大幅提高了开发效率，但不能替代我们的思考和判断

2. **需求能力更重要：**「做什么」比「怎么做」更有价值，想清楚要什么是最大的竞争力
3. **基础是前提：**数据结构、算法、设计模式这些基础知识是驾驭 AI 工具的前提，没基础就判断不了 AI 写的好不好
4. **沟通是关键：**你怎么跟 AI 说话，决定了 AI 输出的质量——好的指令事半功倍
5. **审查是底线：**所有 AI 生成的代码都必须人工检查过才能用，不能盲目信任

对软件工程专业学生的启示

1. **打好基础：**数据结构、算法、操作系统、网络这些基础知识是你的核心壁垒，AI 暂时替代不了
2. **培养设计思维：**从「实现功能」转向「设计系统」，理解怎么划分模块、怎么管理依赖、怎么让系统好扩展
3. **学会给 AI 下指令：**准确描述需求、拆解任务、引导 AI 输出好结果，这是新必备技能
4. **建立审查能力：**能判断 AI 写的东西好不好，发现问题并修正
5. **拥抱变化：**AI 不会取代程序员，但会取代不会用 AI 的程序员
6. **持续学习：**AI 工具在快速迭代，保持学习和适应能力是长期竞争力